



Patricio Whittingslow

He has worked on rocket engines, continuous cell amplification technologies and space computer solder fatigue simulation over his professional career; specializing in the junction of mechanical, electronic and software systems.

Basis of Embedded Microcontroller Development

How do computers interact with the outside world at the lowest level in software- that is the question we'll be answering in this document. We'll start talking about Memory Mapped Registers, how compilers model these at a high level and finally how to turn an LED on using these concepts on a Raspberry Pi Pico board.

Résumé : Patricio nous explique les bases du développement sur micro-contrôleur. Que vous utilisez une Pi Pico, une ESP32 ou tout autre MCU, les fondamentaux sont les mêmes.

The Microcontroller

For the examples in this document we use the **Raspberry Pi Pico** as the platform which has an **RP2040** microcontroller embedded within. You can follow along with a different microcontroller but you'll need to read the datasheet and find your own way. So unless reading and understanding microcontroller unit (MCU) datasheets delights you, get a Pico to follow along as I've already done the boring part of the work for you here! Be sure to get the non-wifi version of the Pico 1 or Pico 2 so that we can turn the LED on and off.

Peripherals

A microcontroller is more than just a CPU. Built into the same chip are specialized hardware units called peripherals — each designed to handle a specific job so the CPU doesn't have to manage every detail itself.

The RP2040 has many peripherals: - GPIO — controlling voltage on the chip's physical pins - UART / I2C / SPI — sending and receiving data over wires - ADC — reading analog values like sensor voltages - PWM — generating timed signals, used for things like motor speed control - Watchdog — automatically resetting the chip if software freezes

Each peripheral is controlled through its own set of memory-mapped registers. The rest of this document is about learning to work those "valves" — starting with GPIO, to turn an LED on.

The Datasheet

You can find the datasheet for the RP2040 online at [raspberrypi.com](https://datasheets.raspberrypi.com/rp2040/). If working with a Pico 2 then the datasheet is the [RP2350](https://datasheets.raspberrypi.com/rp2350/). Datasheets for MCUs contain all information needed to fully operate them.

Memory Mapped Registers

Memory Mapped Registers are the way our software interacts with the physical device. In the case of a microcontroller that means: - Changing the voltage on a pad i.e: GPIO pins and data buses like I2C or UART. - Changing internal functioning to consume more or less energy i.e: low power mode, sleep mode. - Resetting device in certain cases i.e: watchdog timers. - Changing how the CPU works i.e: running software on multiple cores and changing core frequency - Reading voltage or input sensor data i.e: ADC, temperature sensors

Think of interacting with these registers as turning valves that operate a intricate piece of mechanical machinery like a car. The key is coordinating these interactions to get the MCU to do something useful!

OK, so we know what Memory Mapped Registers can help us do- why should we care?

Truth is **every program out there that does something useful interacts with memory mapped registers** at some point. Every software you've ever used. You might not be aware of it but it is most certainly a **basis** of computing.

So why are we not aware of Memory Mapped Registers?

In Software Engineering we choose to hide complexity in order to build reusable, maintainable software. Below is the Go program to turn the Raspberry Pi Pico's LED on:

```
package main
import "machine"
func main() {
    led := machine.LED
    led.Configure(machine.PinConfig{Mode: machine.PinModeOutput})
    led.High()
}
```

Throughout this project we'll actually write the software that does each memory mapped operation individually and by the end we'll have absolute clarity as to why one rarely interacts with memory mapped registers.

We will also give a clear-cut example of a situation where they had to use memory mapped registers to solve real issues like coordinating the stepper motors of a bioresin 3D printer. Let's dive in.

Turning an LED on with memory mapped registers

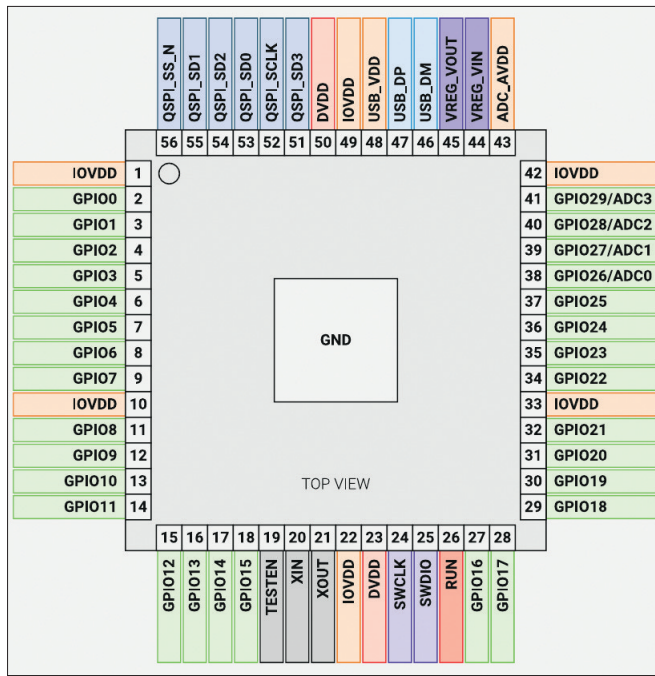
OK, settle in, grab some coffee and open your RP2040 datasheet to page number 11. Here we come across an image of the RP2040 pins available for us to use. You'll note that there are many pins named GPIO, these are pins that we can control freely to turn an LED on and off! **Fig 1**

Below is a pinout diagram of the Raspberry Pi Pico which shows how the GPIOs above are connected to the pins on the board you likely have. GPIO0 is connected to GP0, GPIO1 to GP1, and so on. **Fig2**

So we have the what: we want to control a GPIO's voltage... but HOW do we do that? Well let's start by looking through

the index for "GPIO". Chapter 2 "System Description" contains a section called "GPIO" which may be tempting to start with- however, this GPIO section deals with more complex themes of GPIO control like current draw, impedance and interrupt control. The actual control of the voltage on the pads corresponds to the SIO peripheral of Chapter 2, section 3.1.

Fig 1: Pinout diagram of RP2040 microcontroller.



Chapter 2.3.1 - SIO

The Single-cycle IO block (SIO) is a peripheral on the RP2040 which has the capacity to set the output voltage of GPIOs as we can find out in section 2.3.1.2: GPIO Control- this is exactly what we need to turn an LED on.

In this section we learn that there are three set of GPIO control registers: - Output register: GPIO_OUT and GPIO_HI_OUT, are used to set the output level of the GPIO (1/0 for high/low voltage) - Output enable registers: GPIO_OE and GPIO_HI_OE, are used to enable the output driver. 0 for high impedance, 1 for drive high/low based on GPIO_OUT and GPIO_HI_OUT. High impedance means the pin is disconnected and will not turn on or off. - Input registers: GPIO_IN and GPIO_HI_IN, allow the processor to sample the current state of the GPIOs

Now we know which registers we need to modify we can start investigating how to modify them from software. To modify registers we need to know the address of these registers. Raspberry Pi provides a list of registers and their addresses in section 2.3.1.7 (page 42). From here we learn the SIO registers start at the base address 0xd0000000, 0xd followed by 7 zeros. This is the hexadecimal representation of 3489660928- which means all SIO registers are located at byte position 3489660928 in memory. If we continue reading we'll find out from the list of register table that GPIO_OUT is at offset 0x010 and GPIO_OE is at offset 0x020. These two registers control GPIO outputs for pins 0 through 29 (remember the first picture we saw!). **Fig. 3**

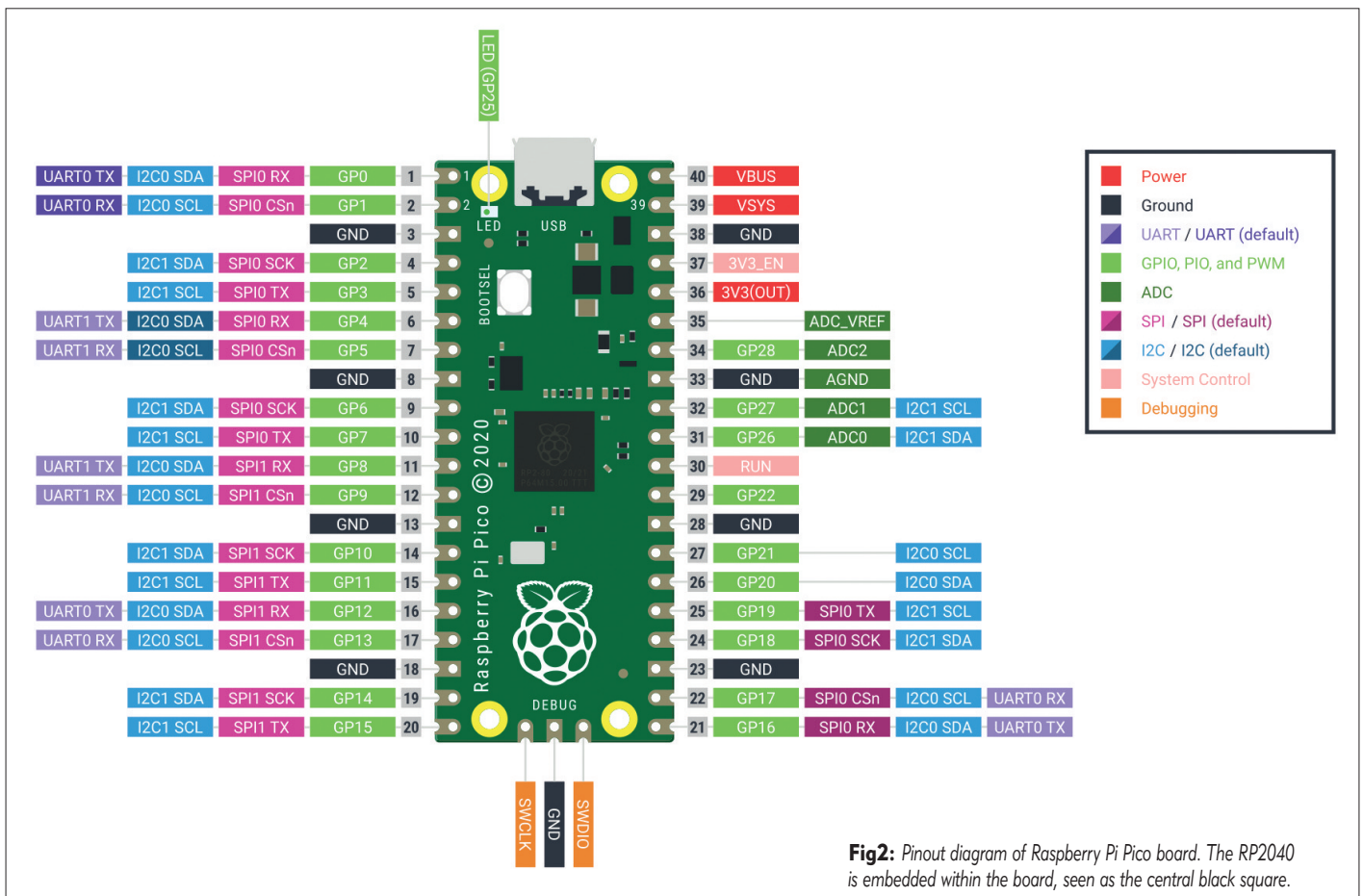


Fig2: Pinout diagram of Raspberry Pi Pico board. The RP2040 is embedded within the board, seen as the central black square.

Chapter 2.19.2 - GPIO, Function Select

We're almost ready to write out our program. If we were to write it with the information we have currently the LED would not turn on though. We are missing crucial information in Chapter 2.19.2, page 237 of the RP2040 datasheet: the function allocated to each GPIO is selected by writing to the FUNCSEL field in the GPIO CTRL register. Like in the previous example we may find the register list with addresses in section 2.19.6.1, page 244: The User Bank IO (IO_BANK0) registers start at a base address of 0x40014000 and the GPIO25_CTRL register we need to control pin 25 (the GPIO pin connected to the LED on the pico!) is at offset 0xc from IO_BANK0. We can generalize the offset of any GPIO_CTRL register by simply doing pin number multiplied by 8- this is because there are 2 consecutive registers per pin and the RP2040's registers are 32 bits (4 bytes) in size.

The program

With all this information laid out: Let's write a Go program that sets these pins!

```
// led.go
package main
import "unsafe"
const ledPin = 25
const ledMask = 1 << ledPin
const addrSIO = 0xd000_0000
const addrGPIO_OUT = addrSIO + 0x010
const addrGPIO_OE = addrSIO + 0x020
const addrIOBank = 0x4001_4000
const sizeOPinBlk = 4 * 2
// There are 2 registers per pin: STATUS and CTRL
// We get the second (CTRL) by adding 4.
const addrGPIO25CTRL = addrIOBank + ledPin*sizeOPinBlk + 4
const fnSIOGPIO = 5

func main() {
    // Attach the SIO to the GPIO.
    writeRegister(addrGPIO25CTRL, fnSIOGPIO)
    // Enable Digital output on that pin in the SIO.
    writeRegister(addrGPIO_OE, ledMask)
    // Turn LED on!
    writeRegister(addrGPIO_OUT, ledMask)
}

func writeRegister(addr uintptr, val uint32) {
    reg := (*uint32)(unsafe.Pointer(addr))
    *reg = val
}
```

The program above is special in that it is designed for running exclusively on the RP2040. Running it on your computer will crash the program with a long error message, likely saying something along the lines of "unexpected fault address 0x400140cc, which is addrGPIO25CTRL's address. To compile and run on the RP2040 we'll need a special compiler, the TinyGo compiler! The following video can teach you how to install and use TinyGo <https://www.youtube.com/watch?v=Fl5eFYU1Xg>.

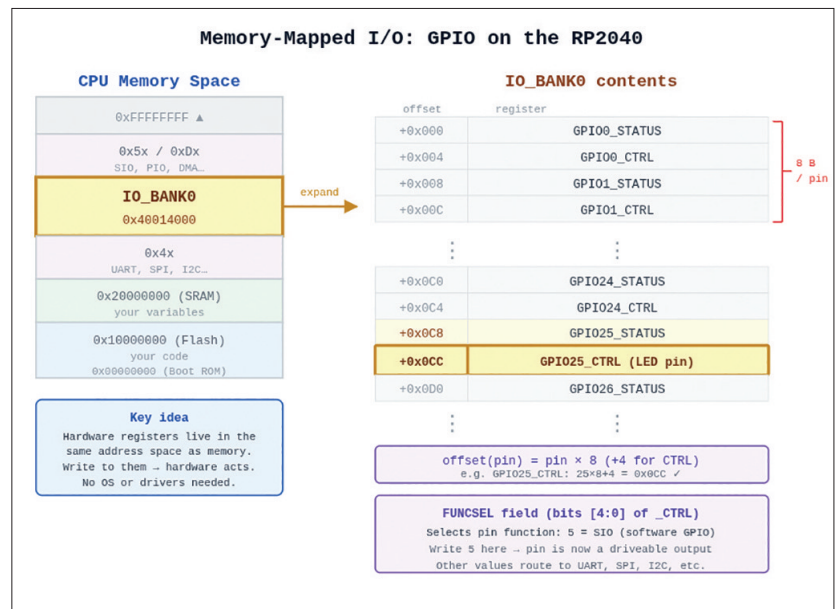


Fig. 3

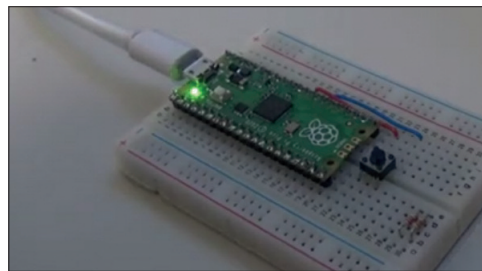


Fig 4:
Raspberry Pi Pico
LED turning on.

```
tinygo flash -target=pico ./led.go
```

You should see your LED turn on!

Fig 4: Raspberry Pi Pico LED turning on.

HAL

So your LED has turned on! Congratulations, if you've made it this far you are in possession of more knowledge of embedded systems than the average embedded systems engineer. This is because a lot of embedded engineers are very accustomed to working with Software Development Kits (SDKs) which are amazing tools to help program these microcontrollers, but alas hide most of the complexity behind these useful abstractions.

SDKs are developed either by the company manufacturing the microcontroller or by the community using it. Designing and developing SDKs is easy! We need to abstract away the hardware using a software layer... a Hardware Abstraction Layer (HAL), if you will.

```
var LED Pin = 25

type Pin uint8

func (p Pin) ConfigOutput() {
    writeRegister(addrIOBank+uintptr(p)*sizeOPinBlk, fnSIOGPIO)
    v := readRegister(addrGPIO_OE)
    writeRegister(addrGPIO_OE, v|(1<<p))
}
```

```
func (p Pin) High() {
    v := readRegister(addrGPIO_OUT)
    writeRegister(addrGPIO_OUT, v|(1<<p))
}

func (p Pin) Low() {
    v := readRegister(addrGPIO_OUT)
    writeRegister(addrGPIO_OUT, v^(1<<p))
}
```

The code above defines a very simple HAL for output pins and provides the LED pin definition. We can then use it to rewrite our program much more simply!

```
package main
func main() {
    LED.ConfigOutput()
    LED.High()
}
```

Wow! We've reduced the amount of lines of code from 24 down to 5! And now no comments are even necessary to explain what is going on... and the programmer does not even need to know about the hardware- whether it is running on a RP2040 or RP2350 or even a STM32 microcontroller! It's as if the software is abstract with respect to which hardware it is running on... Ah! A Hardware Abstraction Layer (HAL)!

It is one of life's greatest joys to find ways to simplify the way humans interface with tools. In this very simple example we hope you've gained appreciation for the work of an embedded engineer. Finding ways to elegantly express hardware ideas through software lets us interact with the world around us more easily and is a way to better understand this curious world that surrounds us.

HAL - Systems Engineering

So remember when we said designing and developing SDKs is easy? Well, maybe it is time for an addendum. A correction to the statement. You see I forgot to mention the SDK we developed with the ConfigOutput and High methods is not quite watertight. As soon as we begin adding more functionality we'd need to take into consideration more systems that come into play. So while it was easy developing it, as we add more systems we'll likely need to change the abstractions we've designed for the hardware. For example: TinyGo provides its very own HAL! This is a HAL we've designed and refined over the years. It is based on Micropython's HAL:

```
package main
import "machine"
func main() {
    led := machine.LED
    led.Configure(machine.PinConfig{Mode: machine.PinOutput})
    led.High()
}
```

It may look a bit more elaborate, but every part serves a purpose to a potential user's needs. In the end it is not much more complicated than our first SDK- although the code for Configure is quite larger than our ConfigOutput.

Real world use case

We promised we'd give examples of the utility of understanding the low level register mapped operations.

This study takes place at Stämm Biotech with the team working on a biocompatible resin 3D printer. The printable space was around 20x20cm and a meter in height, quite large for typical 3D printers. The job was to program the firmware controlling the 2 Z axis motors which lifted the print platform. The requirements for these motors were: - They move at the exact same speed and direction - They move the exact same distance for every operation

And to fulfill these stringent requirements all we really needed to do was to write the following function, all using the register mapped concepts we've seen up to now!

```
// This function moves the 3D printing platform by one step.
func stepMotor(direction bool) {
    const pinStep1, pinStep2, pinDir1, pinDir2 = 1, 2, 3, 4
    const addrGPIO_SET = addrSIO + 0x014
    const addrGPIO_CLR = addrSIO + 0x018
    if direction {
        writeRegister(addrGPIO_SET, (1<<pinDir1)|(1<<pinDir2))
    } else {
        writeRegister(addrGPIO_CLR, (1<<pinDir1)|(1<<pinDir2))
    }
    writeRegister(addrGPIO_SET, (1<<pinStep1)|(1<<pinStep2))
    writeRegister(addrGPIO_CLR, (1<<pinStep1)|(1<<pinStep2))
}
```

Most HALs — Arduino's digitalWrite, MicroPython's Pin.value, even most C SDKs — operate on one pin at a time. To step both motors you'd write something like step(motor1); step(motor2);. That looks simultaneous, but it isn't: there is a small gap between the two calls, and in that gap one motor is already moving while the other hasn't started. Over hundreds of steps, that gap may accumulate into skew, vibration, and eventually a failed print.

Writing a single register that controls both pins collapses that gap to zero — both pins change in the same CPU instruction. This is something no HAL can give you, because the moment you wrap hardware access in a function you've added at least one instruction of latency between operations. Here, the register IS the interface.

Conclusion

We hope your curiosity on the inner workings of microcontrollers has been satisfied. A few things worth taking away: Registers are the boundary between software and the physical world — every program that does something real eventually reaches one, even if it's hidden behind a HAL you've never read.

Abstraction is good engineering. A well-designed HAL lets you focus on what your application does rather than which bits to flip. But abstraction has a cost: the moment you wrap a hardware operation in a function, you've traded away precise timing control. When two stepper motors must move at exactly the same instant, you may choose to reach for the register. You now know what most embedded engineers never need to look at. Use it sparingly, and wisely.